DATABASE NORMALIZATION

A Student Study Guide

Functional Dependency • 1NF • 2NF • 3NF

TM351 — Database Design

PART 1 Functional Dependency

1. Functional Dependency

Functional dependency is the cornerstone concept upon which all normalization is built. Before attempting to normalize any table, you must be able to identify the functional dependencies within it.

Definition

A functional dependency (FD) occurs when the value of one attribute (or a set of attributes) uniquely determines the value of another attribute. Written as: A → B, which reads "A determines B" or "B is functionally dependent on A".

1.1 Notation & Meaning

The notation A → B means:

- Knowing the value of A tells you exactly one value of B.
- A is called the determinant.
- B is called the dependent.

Notation	Reads as	Real Meaning
StudentID → StudentName	"StudentID determines StudentName"	One StudentID maps to exactly one name
CourseID → CourseName, Credits	"CourseID determines CourseName and Credits"	One CourseID gives a unique course name and credit count
StudentName → StudentID	"StudentName determines StudentID"	INVALID — two students can share a name!

1.2 Types of Functional Dependency

There are three main types of functional dependency relevant to normalization:

1 Full (Key) Dependency

A non-key attribute depends on the ENTIRE primary key (all parts). This is the correct, desirable state.

Example: (StudentID, CourseID) → Grade

2 Partial Dependency

A non-key attribute depends on only PART of the primary key (not all of it). This violates 2NF and must be removed.

Example: StudentID → StudentName (only part of the compound PK)

3

Transitive Dependency

A non-key attribute depends on ANOTHER non-key attribute (not directly on the PK). This violates 3NF and must be removed.

Example: EmpNo → DeptNo → DeptName (DeptName depends on DeptNo, not EmpNo directly)

PART 2 Normalization — Overview

2. What is Normalization?

Normalization is the systematic process of organizing the attributes and tables of a relational database to reduce redundancy and improve data integrity. It applies a series of rules — called Normal Forms — each building upon the previous.

> **The Golden Rule of Normalization**
>
> *"Every non-key attribute must depend on the key, the whole key, and nothing but the key."*

2.1 Why Normalize?

Without normalization, databases suffer from three types of anomalies:

Anomaly	What Goes Wrong	Example
Update Anomaly	Changing one fact requires updating many rows; missing even one causes inconsistency.	Employee moves department — must update every row with their name.
Insert Anomaly	Cannot record certain facts without also recording unrelated facts.	Cannot add a new department until at least one employee is assigned.
Delete Anomaly	Deleting a row unintentionally destroys other useful facts.	Deleting the last employee in a department loses all department information.

2.2 The Normal Forms at a Glance

Normal Form	Requirement	Problem It Fixes
UNF (Unnormalized)	Raw data — may contain repeating groups and multiple values in one cell.	Starting point only.
1NF (First)	Atomic values only; no repeating groups; each row is unique.	Removes multi-valued and repeating attributes.
2NF (Second)	In 1NF + no partial dependencies (every non-key attr depends on the full PK).	Removes partial dependencies (only applies to compound keys).
3NF (Third)	In 2NF + no transitive dependencies (non-key attrs depend only on the PK).	Removes transitive dependencies.

PART 3 First Normal Form (1NF)

3. First Normal Form (1NF)

1NF Rule

A table is in First Normal Form if: • Every column contains atomic (indivisible) values. • There are no repeating groups or multi-valued attributes. • Each row is uniquely identifiable (has a primary key).

3.1 How to Achieve 1NF — Step by Step

1. Identify any column that stores multiple values or a repeating group.
2. Move repeating attributes to a new table.
3. Copy the original primary key into the new table.
4. Create a compound primary key in the new table (original PK + new attribute).

3.2 Violation Detection

Look for these warning signs in a table:

- A column named "Items", "Products", "Courses" that holds comma-separated or listed values.
- Columns like Course1, Course2, Course3 (numbered duplicates — also a repeating group).
- A cell containing "Math, Physics, Chemistry" — not atomic!

⚠ Common Mistake

A column with a single value that just happens to repeat across rows is NOT a 1NF violation. For example, "City = Muscat" appearing in many rows is fine — the value itself is atomic. A violation occurs when a single cell holds multiple values.

PART 4 Second Normal Form (2NF)

4. Second Normal Form (2NF)

2NF Rule

A table is in Second Normal Form if: • It is already in 1NF. • Every non-key attribute is fully functionally dependent on the ENTIRE primary key. (No partial dependencies allowed.)

Important Note

2NF only applies when the table has a COMPOUND primary key (two or more attributes). If the primary key is a single attribute, the table is automatically in 2NF once it is in 1NF, because there is no "part" of the key to partially depend on.

4.1 How to Achieve 2NF — Step by Step

5. Verify the table is in 1NF.
6. Identify all partial dependencies: non-key attributes that depend on only part of the compound PK.
7. For each partial dependency, create a new table with: — The partial key (the part of the PK it depends on) as the new PK. — The dependent attributes.
8. Remove those attributes from the original table.
9. The original table keeps only the full compound PK and attributes that fully depend on it.

4.2 Identifying Partial Dependencies

For each non-key attribute, ask: "Could I determine this attribute using only part of the primary key?"

- If YES → it is a partial dependency → must be moved.
- If NO → it fully depends on the whole key → keep it.

PART 5 Third Normal Form (3NF)

5. Third Normal Form (3NF)

3NF Rule

A table is in Third Normal Form if: • It is already in 2NF. • No non-key attribute is transitively dependent on the primary key. (Non-key attributes must NOT depend on other non-key attributes.)

5.1 What is a Transitive Dependency?

A transitive dependency exists when: A → B → C

- A is the primary key.
- B is a non-key attribute.
- C is another non-key attribute that depends on B, not directly on A.

Even though A → C is technically true (through B), C is transitively dependent, not directly dependent.

5.2 How to Achieve 3NF — Step by Step

10. Verify the table is in 2NF.
11. Look for chains: PK → non-key attribute → another non-key attribute.
12. Create a new table with: — The middle non-key attribute as the new PK. — The transitively dependent attributes.
13. In the original table, keep the middle attribute as a Foreign Key but remove the transitively dependent attributes.

⚠ Transitive vs. Partial

Partial dependency: non-key attribute depends on PART of the PK. Transitive dependency: non-key attribute depends on ANOTHER non-key attribute. These are different problems requiring different fixes!

PART 6 Illustrated Examples — UNF → 1NF → 2NF → 3NF

EXAMPLE 1 | Hospital Patient Records

Scenario

A hospital stores patient treatment data in one flat table. The table records which doctor treated which patient on which date, along with the diagnosis.

Step 1 — Unnormalized Form (UNF)

Primary Key candidate: {PatientID, DoctorID, TreatmentDate}

PatientID	PatientName	City	DoctorID	DoctorName	Specialty	TreatmentDate	Diagnosis
P01	Fatma	Muscat	D10	Dr. Ali	Cardiology	2025-01-05	Hypertension
P01	Fatma	Muscat	D10	Dr. Ali	Cardiology	2025-03-12	Flu
P02	Yusuf	Salalah	D11	Dr. Hana	Neurology	2025-02-20	Migraine
P03	Maha	Muscat	D10	Dr. Ali	Cardiology	2025-04-01	Anemia

UNF Problems

No repeating groups — this table is already technically in 1NF since each cell holds a single atomic value and there are no multi-valued attributes. However, it has serious partial and transitive dependency problems that must be resolved in 2NF and 3NF.

Step 2 — First Normal Form (1NF)

Check: Are all values atomic? Yes. Are there repeating groups? No. Is every row unique? Yes (PatientID + DoctorID + TreatmentDate).

Result: This table is already in 1NF. The compound primary key is {PatientID, DoctorID, TreatmentDate}.

Step 3 — Second Normal Form (2NF)

Identify partial dependencies — attributes that depend on only part of the compound PK:

Attribute	Depends on...	Type	Action
PatientName	PatientID only	Partial dependency	Move to Patient table
City	PatientID only	Partial dependency	Move to Patient table

Attribute	Depends on...	Type	Action
DoctorName	DoctorID only	Partial dependency	Move to Doctor table
Specialty	DoctorID only	Partial dependency	Move to Doctor table
Diagnosis	PatientID + DoctorID + TreatmentDate (all)	Full dependency	Keep in main table

After removing partial dependencies, the 2NF tables are:

PatientID 🔑	PatientName	City
P01	Fatma	Muscat
P02	Yusuf	Salalah
P03	Maha	Muscat

Patient Table (PK = PatientID)

DoctorID 🔑	DoctorName	Specialty
D10	Dr. Ali	Cardiology
D11	Dr. Hana	Neurology

Doctor Table (PK = DoctorID)

PatientID 🔑	DoctorID 🔑	TreatmentDate 🔑	Diagnosis
P01	D10	2025-01-05	Hypertension
P01	D10	2025-03-12	Flu
P02	D11	2025-02-20	Migraine
P03	D10	2025-04-01	Anemia

Treatment Table (PK = PatientID + DoctorID + TreatmentDate)

Step 4 — Third Normal Form (3NF)

Check each 2NF table for transitive dependencies:

- Patient Table: PatientID → PatientName, City. No non-key attribute depends on another non-key attribute. ☑ Already in 3NF.
- Treatment Table: All attributes (Diagnosis) depend directly on the full compound PK. ☑ Already in 3NF.
- Doctor Table: DoctorID → DoctorName, Specialty. No transitive dependency here. ☑ Already in 3NF.

Final 3NF Result — Example 1

Three tables: • Patient (PatientID 🔑, PatientName, City) • Doctor (DoctorID 🔑, DoctorName, Specialty) • Treatment (PatientID 🔑 FK, DoctorID 🔑 FK, TreatmentDate 🔑, Diagnosis)